❹ Extracts the layers from the archive applying the layered-JAR mode

❺ OpenJDK base image for the second stage

❻ Copies each JAR layer from the first stage to the second stage inside the “workspace” folder

❼ Uses the Spring Boot Launcher to start the application from the layers rather than an uber-JAR

NOTE What if you want to change the configuration for the layers in the JAR file? As always, Spring Boot provides sensible defaults, but you can customize it and adapt it to your needs. Perhaps your project has internal shared dependencies you might want to place in a separate layer, since they change more often than third-party dependencies. You can do that through the Spring Boot Gradle or Maven plugin. For more information, refer to the Spring Boot documentation at https://spring.io/projects/spring-boot.

The process for building and running the container is the same as before, but now the image is more efficient and is optimized at build and execution time. However, it’s still not ready for production. What about security? That’s the topic of the next section.

SECURITY

Security is a critical aspect that is too often underestimated by people getting started with Docker and containerization. You should be aware that containers run using the root user by default, potentially letting them get root access to the Docker host. You can mitigate the risk by creating a non-privileged user and using it to run the entry-point process defined in the Dockerfile, following the principle of least privilege.

Consider the Dockerfile you wrote for Catalog Service. You can improve it by adding new steps to create a new non-root user that will run the application.

Listing 6.4 More secure Dockerfile to build a Catalog Service image

```
FROM eclipse-temurin:17 AS builder
WORKDIR workspace
ARG JAR_FILE=build/libs/*.jar
COPY ${JAR_FILE} catalog-service.jar
```